Aprenentatge Computacional

Anàlisi i Machine Learning en la Predicció del Guanyador de Partides d'Escacs de Lichess

Inés Gómez Carmona (1704361), Naroa Sarrià Gil (1706167)

Github Cas Kaggle - Chess

Abstract—Aquest treball s'analitza un gran conjunt de partides d'escacs, on s'ha portat a terme la resolució d'un problema de classificació supervisada amb les dades de milers de partides de la pàgina de Lichess.org amb l'objectiu de predir el guanyador d'una partida a partir de diverses característiques dels jugadors i de les partides.
El procés inclou una anàlisi exploratoria de les dades per entendre l'estructura, un preprocessament per adaptar-les als nostres models de predicció i l'entrenament i comparació de diversos models de classificació: la *Regressió Logística, el Support Vector Machine, el Gradient Boosting i el Random Forest* avaluant amb les mètriques corresponents, aplicant tècniques de validació creuada i cerca d'hiperparàmetres per avaluar el nostre projecte de machine learning.
D'aquesta manera s'ha pogut avaluar com evoluciona la capacitat predictiva a mesura que es coneixen més moviments de la partida i, per tant, analitzar detalladament els resultats obtinguts durant l'aprenentatge dels diferents models per treure les conclusions adients.

Contents

1 Introducció 1
2 Exploratory data analysis 1
3 Preprocessing i Feature Engineering 2
3.1 Codificació i tractament inicial de variables 2
3.2 Outliers 2
3.3 Creació de noves variables 2
3.3.1 Variables prèvies a l'inici de la partida 2
3.3.2 Variables generades durant la partida 3
3.4 Correlacions 3
3.4.1 Dades prèvies a l'inici de la partida 3
3.4.2 Dades en el transcurs de la partida 3
3.5 Normalització de les dades 3
4 Metric selection 4
4.1 Mètriques numèriques 4
4.2 Mètriques gràfiques 4
5 Selecció de models i validació creuada 4
6 Anàlisi final 5

1. Introducció

En aquest treball s'analitza un gran conjunt de partides d'escacs extret de la plataforma Lichess.org, amb l'objectiu d'abordar un problema real de classificació binària i supervisada. Així, l'objectiu és predir quin jugador guanya la partida a partir d'un conjunt de característiques conegudes dels jugadors i de la seqüència de moviments al llarg de la partida.
Cada instància del dataset correspon a una partida completa i inclou informació sobre la identificació dels jugadors, el seu rating, el color assignat, la llista de moviments... i segons quin jugador ha guanyat, la variable objectiu ens indica el color de fitxes del jugador victoriós.
Com hem dit, l'objectiu principal del projecte és construir un model capaç de predir el guanyador en funció tant d'informació prèvia dels jugadors com d'aspectes dinàmics derivats de l'estat del tauler. Per això, hem fet un procés exhaustiu de feature engineering per generar noves variables (per exemple càlcul d'històrics de victòries, estadístiques entre parelles de jugadors, ratxes positives i negatives, i mesures derivades de l'estat del taulell després d'un nombre determinat de moviments).
Un cop preprocessades les dades i generades les noves característiques, s'han entrenat diversos models de classificació (Regressió Logística, Support Vector Machine, Random Forest i Gradient Boosting), aplicant validació creuada i tècniques bàsiques d'avaluació per comparar el rendiment. Finalment, s'ha analitzat com evoluciona la capacitat predictiva del model a mesura que es coneixen més moviments de la partida, estudiant el comportament dels models per tal d'extreure les conclusions adients.

2. Exploratory data analysis

Començarem analitzant i entenent els atributs de la nostra base de dades de partides d'escacs *(Chess Game Dataset (Lichess))*. Veiem que tenim 16 atributs incials *(id, rated, created_at, last_move_at, turns, victory_status, winner, increment_code, white_id, white_rating, black_id, black_rating, moves, opening_eco, opening_name, opening_ply)*, que ens aporten tant informació dels jugadors com del desenvolupament de la partida.
La variable *id* identifica de manera única cada partida, mentre que *rated* indica si és valorada i afecta el rànquing dels jugadors o no. Els camps *created_at* i *last_move_at* corresponen als temps d'inici i de final del joc, i *turns* recull el nombre total de jugades realitzades per part dels dos jugadors (semijugades). L'etiqueta *victory_status* indica la forma en què ha acabat la partida (per mat, temps, abandonament…), i *winner* especifica el color guanyador. La variable *increment_code* descriu el control de temps emprat. Els atributs *white_id* i *black_id* identifiquen els jugadors, mentre que *white_rating* i *black_rating* contenen el seu ELO inicial. El camp *moves* inclou tota la seqüència de moviments en notació algebraica oficial d'escacs, i, finalment, les variables *opening_eco, opening_name i opening_ply* descriuen el codi, el nom i la durada de l'obertura jugada. Aquest conjunt d'informació serveix com a punt de partida per a l'anàlisi exploratòria i la posterior generació de noves variables explicatives.
Ara analitzem el nostre target, és a dir, el guanyador de la partida, per veure la seva distribució inicial. Els seus possibles valors a prendre són *white, black, draw* en funció de si guanya el jugador de blanques, de negres, o la partida acaba en empat. D'aquesta manera podem determinar si el dataset està balancejat o no.

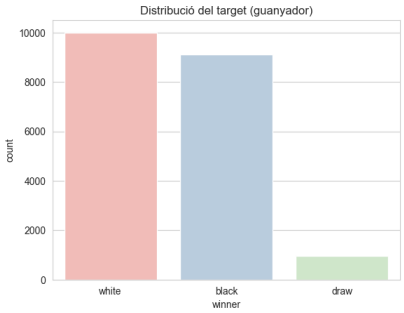

Figure 1. Distribució del target inicial

Podem observar que un 49.86% de victòries corresponen a les blanques, un 45.40% correspon a guanyadors amb negres i només un 4.74% correspon a partides que acaben en empat. Veient aquests resultats s'ha decidit extreure de la base de dades les instàncies de partides acabades en empats, ja que corresponen a menys d'un 5%, només aportarien soroll a les dades i ens complicarien la part d'entrenar els models de classificació. D'aquesta manera aconseguim simplificar els nostres futurs models realitzant classificació binària. D'aquesta manera, la distribució final ens quedarà així:

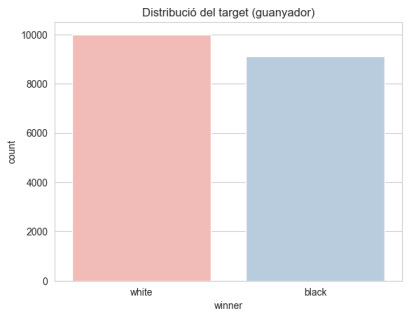

Figure 2. Distribució del target white vs black

On veiem que tenim un 52.34% de victòries corresponents a blanques i un 47.66% corresponents a negres. Podem dir, per tant que el nostre dataset està balancejat.

Revisarem també la distribució del ratings, la variable que ens sembla més rellevant a l'hora de predir el guanyador de la partida.

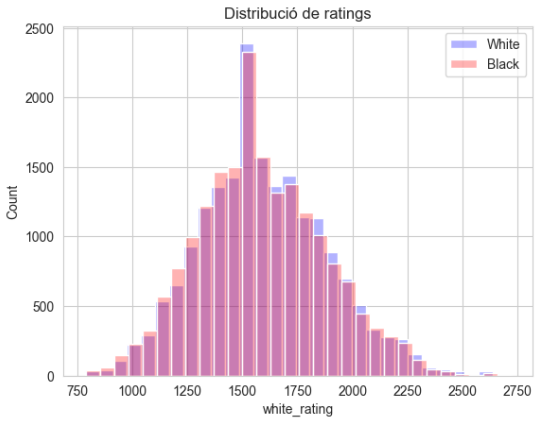

Figure 3. Distribucions dels ratings de jugadors

Veiem que la distribució dels ratings presenta una forma aproximadament gaussiana, amb un pic al voltant dels 1500 punts d'ELO. Aquest comportament és esperable en jugadors aficionats i semicompetitius, ja que la majoria es concentren en la franja central. Podem dir, per tant, que la distribució dels ratings és equilibrada entre colors, fet que evitarà induir biaixos als models. Així, la majoria de les partides involucren jugadors amb un ELO aproximat d'entre 1200 i 2000, mentre que els valors inferiors a 1000 i superiors a 2300 són molt menys freqüents. És a dir, tenim el dataset dominat per jugadors intermedis.

Finalment, en aquest apartat analitzem el camp *opening_eco*, que conté el codi normalitzat ECO (*Encyclopaedia of Chess Openings*), ja que resulta molt més manejable que la variable *opening_name*, on teniem més d'un miler de noms diferents d'obertures úniques. Per tant, estudiant *opening_eco* trobem 359 codis ECO únics. Els codis més freqüents són per exemple A00 (968 aparicions), C20 (806)... I hi ha d'altres que només apareixen una sola vegada. Això ens motiva a, posteriorment, agrupar les obertures segons la seva lletra inicial (A, B, C, D, E), ja que aquesta classificació ja segueix l'estructura oficial de l'ECO i redueix significativament la dimensionalitat. Això permetrà treballar amb una variable categòrica més neta, interpretable i útil per als models de classificació, evitant el soroll introduït pels nombrosos casos poc representatius.

3. Preprocessing i Feature Engineering

Abans de començar a realitzar models i entrenar-los s'ha de dur a terme una neteja i preparació de les dades per així, assegurar-nos de què siguin aptes pels algorismes de classificació. D'altra banda, també era necessari utilitzar tècniques de feature engineering per tal de generar i seleccionar característiques rellevants per millorar el rendiment dels models. D'aquesta manera, en aquest apartat descrivim els processos de preparació i creació de les variables explicatives adequades.

3.1. Codificació i tractament inicial de variables

Abans de generar noves caraterístiques, es va revisar el dataset original per detectar possibles valors nuls presents. Es va veure, per tant, que les dades no presentaven valors NaN's originals, de manera que, posteriorment, només vam haver de centrar-nos a no generar-ne nosaltres a l'hora de generar les noves característiques, cosa que s'explica detalladament a l'apartat adient.

La variable target *(winner)*, que originalment era categòrica *("white", "black")* es va codificar en forma binària assignant els valors *1 - victòria de les blanques; 0 - victoria de les negres*.

D'altra banda, tal com s'ha especificat a l'apartat anterior, la classificació de les obertures *(opening_eco)* s'han classificat segons el seu tipus, obtenint la classificació final següent:

- *A - Obertures de flanc i poc comunes*
- *B - Obertures semiobertes*
- *C - Obertures obertes*
- *D - Obertures tancades i semitancades*
- *E - Obertures amb defenses índies*

3.2. Outliers

Observant les nostres variables, només s'ha considerat necessari analitzar outliers en la variable de *turns*. Primerament, s'ha posat un llindar de 4 semijugades, ja que aquest és el nombre mínim de moviments (2 per jugador) per poder finalitzar una partida. Així, totes les instàncies amb un valoro inferior a aquest llindar s'han eliminat perquè no representen partides reals i aportarien soroll més endavant. També s'ha establert un llindar superior de 200 moviments, pel fet que són casos molt extrems i molt escassos.

Pel que fa a la resta de variables, no s'han eliminat més possibles outliers, ja que es considera que aquests possibles valors "extrems" formen part de la naturalesa del joc. Per exemple diferències molt elevades de *rating* o jugadors amb molta experiència són situacions realistes que els models hauran de ser capaços de processar i no provenen en cap cas d'errors de mesura ni de registres equivocats.

3.3. Creació de noves variables

La generació de noves característiques ha estat una part molt important del projecte. Així, s'han creat dos grans blocs de variables: (1) les que depenen només de la informació prèvia a l'inici de la partida, i (2) les que s'obtenen reconstruint el tauler durant els primers N moviments.

3.3.1. Variables prèvies a l'inici de la partida

Aquestes característiques es van calcular recorrent seqüencialment i, havent ordenat cronològicament totes les partides, per tal de simular l'evolució temporal dels jugadors. Així, s'han generat les següents variables:

- **Diferència de rating**: la variable *diff_rating* es va crear per tal de quantificar la distància entre els nivells dels jugadors que s'enfronten a cada partida, la qual hauria de ser molt més significativa que les variables dels ratings de cada jugador per separat.
- **Winrates globals**: a partir de l'historial acumulat de cada jugador, es va calcular el seu percentatge de victòries fins al moment. D'aquesta manera per cada instància tindrem les noves variables de *white_global_winrate, black_global_winrate i diff_global_winrate*, on l'última ens indica la diferència entre els winrates globals dels dos jugadors.
- **Històric entre parelles**: per cada partida es van generar variables per indicar quantes vegades la parella de jugadors s'havien enfrontat anteriorment, i quantes d'aquestes partides anteriors havia guanyat el jugador de blanques. Així vam crear les variables explicatives de *vs_white_winrate i vs_prev_matches*.

- **Experiència**: observant la nostra base de dades, vam veure que teníem molts jugadors recurrents, cosa que ens va motivar per crear noves variables amb l'experiència de cada jugador. Es va registrar el nombre total de partides prèvies de cada jugador (*white_experience*, *black_experience*), així com la diferència entre ambdós (*diff_experience*).
- **Ratxes positives i negatives**: es va implementar un sistema de ratxa que compta victòries consecutives com a valors positius i derrotes consecutives com a valors negatius, generant quatre variables (*white_win_streak*, *white_loss_streak*, *black_win_streak*, *black_loss_streak*).

Aquest conjunt de noves variables explicatives ens aporta més informació sobre l'historial passat dels jugadors i sobre el seu rendiment a mesura que avança el temps.

3.3.2. Variables generades durant la partida

Per obtenir característiques a partir dels moviments a mesura que avança la partida, s'ha utilitzat la llibreria *python-chess*, que permet reconstruir el tauler a partir de la seqüència de moviments anotats. Fixat un nombre de moviments N, per a cada partida es reconstrueix l'estat del tauler després dels primers N plies i es calculen un conjunt de variables que expliquem a continuació:

- **Nombre de peces restants**: variables *white_piece_count* i *black_piece_count*, que ens indiquen quantes peces té cada color després dels N primers moviments de la partida.
- **Valor material disponible**: variables que ens indiquen la suma del valor clàssic de les peces de cada jugador (*white_material_value*, *black_material_value*), utilitzant valors estàndard (peó=1, cavall=3, alfil=3, torre=5, dama=9).
- **Balanç de material**: variable *material_balance*, definida com la diferència entre el material de les blanques i el de les negres.
- **Control del centre**: recompte de quantes peces de cada color ataquen les caselles centrals D4, E4, D5 i E5. (*white_center_control*, *black_center_control*).
- **Mobilitat**: nombre de moviments legals que pot fer cada jugador en la posició del tauler actual (*white_mobility*, *black_mobility*).
- **PST (Piece–Square Tables)**: variables amb les puntuacions *white_PST* i *black_PST* que s'obtenen sumant els valors de taules PST clàssiques, especialment per a peons i cavalls. Aquestes taules assignen un valor diferent a cada casella, reflectint les preferències estratègiques com el control del centre o la proximitat a zones actives.
- **Space**: variables que quantifiquen el control espacial de cada color en la meitat del tauler de l'oponent. Així, *white_space* i *black_space* compten quantes caselles de la meitat rival estan atacades per cada color, mesurant d'aquesta manera el domini territorial.

Amb aquest conjunt de variables podem obtenir informació posicional i estratègica important per avaluar l'estat de la partida donats uns certs moviments.

3.4. Correlacions

A continuació passarem a analitzar dues matrius de correlacions, la primera només amb les variables prèvies a l'inici de la partida i l'altra amb les variables creades a partir del començament de la partida.

3.4.1. Dades prèvies a l'inici de la partida

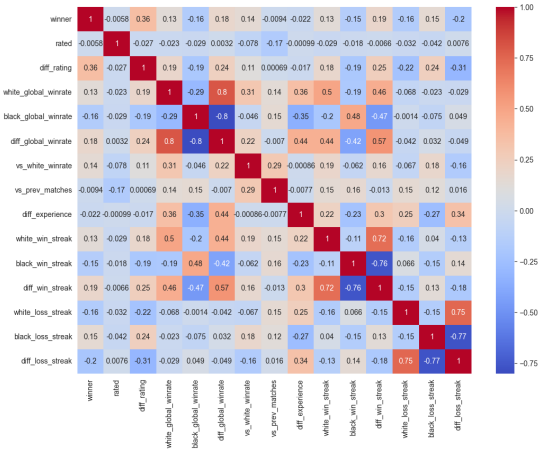

Figure 4. Matriu de correlacions de variables prèvies a la partida

D'aquesta matriu ens interessa analitzar la correlació de cada variable amb la variable objectiu. On clarament veiem que la variable *diff_rating* és la que té una correlació més alta. Podem veure també que variables com *rated*, *vs_prev_matches* i *diff_experience* tenen una correlació molt baixa, però no podem descartar-les totes, ja que la variable *vs_prev_matches* la necessitem juntament amb la variable *vs_white_winrate* per distingir els casos en què dos jugadors no s'han enfrontat mai dels casos en què el jugador de les blanques mai ha guanyat. De les parelles de variables del global winrate, les ratxes de victòries i les ratxes de derrotes, aconseguim una correlació més alta si fem servir la seva diferència, és a dir, les variables *diff_global_winrate*, *diff_win_streak* i *diff_loss_streak*.

3.4.2. Dades en el transcurs de la partida

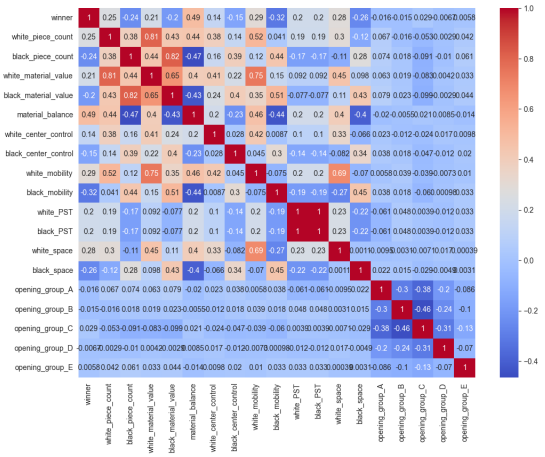

Figure 5. Matriu de correlacions de variables generades posteriors al començament

En aquesta matriu observem com totes les variables dels grups d'obertures tenen una correlació molt poc significativa amb la variable objectiu, de manera que totes aquestes variables les descartem. Passem a analitzar la resta de variables. Veiem que tots els coeficients de correlació entre les variables explicatives i la variable objectiu totes tenen un valor absolut més gran que 0.14, aquest valor no és molt elevat, però vol dir que existeix una relació entre aquestes variables. Observant que el major coeficient que tenim és de 0.49 (*material_balance*), hem decidit deixar tota la resta de variables, ja que cap té una correlació molt alta, però totes indiquen que hi tenen relació amb la variable objectiu, la qual cosa ens interessa.

3.5. Normalització de les dades

Finalment, amb les variables explicatives escollides s'ha realitzat una normalització de numèriques aplicant *StandardScaler (mitjana 0 i desviació 1)*, per assegurar-nos de tenir-les totes en el mateix rang.

4. Metric selection

Ara analitzarem quina és la mètrica de classificació més adient pel nostre problema. Abans d'entrenar models més complexos, hem d'establir quines mètriques utilitzarem per avaluar-ne el rendiment.

4.1. Mètriques numèriques

Analitzant el nostre dataset, veiem que el nostre dataset processat està bastant balancejat. Tenim un 52.34% de victòries corresponents a blanques i un 47.66% corresponents a negres. Així, tenim que la clàssica mètrica *l'accuracy_score* (encerts globals) és un bon indicador del rendiment global dels nostres models.

Cal esmentar, però, que s'ha generat una funció per visualitzar totes les mètriques anteriors mitjançant `classification_report()`, per poder analitzar posteriorment l'evolució de totes en conjunt, tot i que prendre'm l'accuracy com a referència.

4.2. Mètriques gràfiques

D'altra banda, per analitzar visualment el comportament dels nostres models, hem generat una suite de funcions gràfiques que mostren les dues corbes principals:

- La corba *ROC* ens relaciona la taxa de vertaders positius (TP) amb la de falsos positius (FP). Hem afegit el valor de l'àrea sota la corba (AUC), que ens serveix com indicador per saber com és de bona la gràfica. Com més proper estigui el valor a 1, millor és el model.
- La corba *Precision-Recall* ens mostra la relació entre la precisió i el recall per a diferents llindars de decisió. En aquesta hem afegit el valor de l'Average Precision Score (AP), que representa l'àrea sota la corba Precision-Recall i resumeix el seu comportament global.

En el nostre context, considerem que és més rellevant la corba *ROC*, ja que ja ens reflecteix el rendiment sobre la classificació del guanyador, que, com ja hem esmentat, està balancejada.

A més a més, juntament amb aquestes dues gràfiques, hem afegit la matriu de confusió, per visualitzar numèricament com de bé classifiquen els nostres models finals.

5. Selecció de models i validació creuada

Com a models, hem considerat la Regressió Logística, que pot ser útil com a model inicial, el Support Vector Machine, que pot ser adequat quan la separació de classes no és evident i el Gradient Boosting i el Random Forest, que són models basats en arbres de decisió.

Primer de tot, vam entrenar els models amb els hiperparàmetres per defecte fixant el nombre de moviments N = 0, de manera que només s'utilitzaven les variables prèvies de la partida. Vam observar que tots tenien un accuracy d'aproximadament un 0.66, excepte el Random Forest, que es quedava amb un 0.62. Tot i això, vam decidir no descartar cap model encara, ja que, en mètodes basats en arbres de decisió, els hiperparàmetres poden tenir un impacte significatiu en el rendiment i llavors vam decidir que havíem d'incloure Random Forest per la cerca d'hiperparàmetres per comprovar si millorava.

Per fer la cerca d'hiperparàmetres hem aplicat una validació creuada amb un K-Fold de 5 particions i hem utilitzat la funció RandomizedSearchCV, de manera que no es proven totes les possibilitats, però computacionalment és més assequible.

Els resultats que obtenim una vegada aplicada la cerca d'hiperparàmetres òptims són els següents:

Table 1. Resultats posteriors a la cerca d'hiperparàmetres

Model	Test accuracy	AUC
Regressió Logistica	0.6621	0.74
SVM	0.6660	0.74
Gradient Boosting	0.6642	0.74
Random Forest	0.6647	0.74

Observem que els resultats són pràcticament els mateixos per cada model, de manera que concloem que el model amb N = 0 no es pot millorar més i amb qualsevol model que utilitzem obtenim un accuracy del 66%.

Veient aquests resultats, s'ha de remarcar que el fet d'obtenir un 66% d'accuracy amb $N = 0$ no resulta sorprenent. Amb només la informació prèvia dels jugadors, com els ratings, winrates o les ratxes dels jugadors, els models ja disposen d'un conjunt de característiques altament informatives. Aquestes característiques, i sobretot la diferència dels ratings, ja reflecteixen les diferències de nivell prou marcades entre jugadors. Per tant, amb aquestes variables ja s'aconsegueix encertar aproximadament dues de cada tres partides, i és esperable que la millora obtinguda amb la cerca d'hiperparàmetres en aquest cas no millori més els resultats.

Ara, a partir d'afegir informació posicional, derivada dels moviments inicials, i a mesura que avança la partida, quan els models podran captar patrons més profunds i millorar significativament el seu rendiment. Per tant, passem a veure com millora progressivament el model afegint les variables que tenen en compte els N primers moviments de la partida.

Primer de tot, entrenem els quatre models amb els hiperparàmetres per defecte amb N = opening_ply i observem que obtenim resultats semblants als anteriors: la Regressió Logística va obtenir un accuracy de 0.668, Gradient Boosting 0.670, SVM 0.653 i Random Forest 0.647. Tots són bastant similars entre si, però decidim descartar el Support Vector Machine (SVM), ja que el temps d'execució és significativament més elevat que la resta de models i, per tant, no és gens eficient.

Una vegada descartat el SVM entrenem els altres tres models amb diferents nombres de moviments N i obtenim els següents resultats:

Table 2. Accuracy obtingut amb els hiperparàmetres per defecte

N	L.Regression	G.Boosting	R.Forest
0	0.6501	0.6629	0.6233
10	0.6960	0.6625	0.6482
20	0.6463	0.6811	0.6803
30	0.6119	0.6959	0.6928
40	0.6226	0.7240	0.7302
50	0.6072	0.7696	0.7714
60	0.6114	0.7845	0.7857
80	0.5936	0.8152	0.8232
100	0.5508	0.8533	0.8578
120	0.5412	0.9021	0.9227
150	0.5106	0.9787	0.9787

En aquesta taula podem veure que la Regressió Logística empitjora a mesura que augmentem el nombre de moviments. Això pot ser perquè quan augmentem el nombre de moviments utilitzats, les relacions entre les variables i la variable objectiu són massa complexes pel model i aquest no té un bon rendiment. En canvi, podem veure com Gradient Boosting i Random Forest tenen resultats semblants, que van millorant a mesura que augmentem el nombre de moviments. D'aquesta manera, descartem el model de Regressió Logística.

A continuació, farem la cerca d'hiperparàmetres i la validació creuada. No farem per totes les N d'abans, sinó que escollirem uns en concret. La mitjana de moviments és de 60, per tant, volem utilitzar un nombre menor a aquest, ja que, si fem servir una N molt propera a 60, és probable que moltes de les partides estiguin a punt d'acabar, de manera que predir el resultat serà trivial. En conseqüència, usarem N = 35, per tal d'assegurar-nos que la majoria de les partides no estan a punt d'acabar.

Ara, fem la cerca d'hiperparàmetres amb un K-Fold de 5 particions i obtenim que els millors paràmetres pel Gradient Boosting són `{subsample: 0.6, n_estimators: 200, min_samples_split: 15, min_samples_leaf: 10, max_features: 'log2', max_depth: 4, learning_rate: 0.05}` i pel Random Forest són `{n_estimators: 300, min_samples_split: 10, min_samples_leaf: 1, max_features: 'sqrt', max_depth: 20, bootstrap:`

`True}`. Els resultats obtinguts amb aquests hiperparàmetres són els següents:

Table 3. Resultats després de la cerca d'hiperperàmetres

Model	Accuracy	AUC
Gradient Boosting	0.7363	0.81
Random Forest	0.7376	0.81

Amb tots dos models obtenim resultats pràcticament iguals, de manera que és indiferent quin dels dos models seleccionem. D'aquesta manera, decidim quedar-nos amb Gradient Boosting. Executem ara amb els hiperparàmetres obtinguts el model per diferents N i veiem com ens queda la gràfica del accuracies obtinguts.

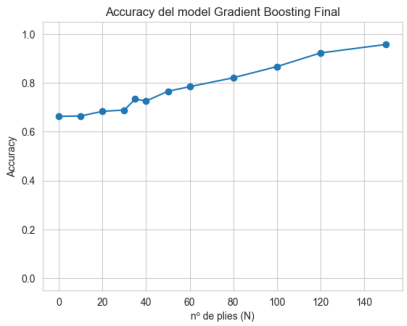

Figure 6. Accuracies amb el model final

Finalment, tanquem aquest apartat comentant que, a partir dels $N \approx 30$ observem una millora significativa, i per valors més grans, l'accuracy arriba a superar el 90%. Tot i que això remarca la potència i importància de les característiques posicionals creades, cal tenir en compte que utilitzar valors massa elevats de N pot fer que les prediccions siguin gairebé trivials. Això és degut al fet que algunes partides poden estar pràcticament decidides.

6. Anàlisi final

Recapitulant, en aquest projecte s'ha abordat el problema de predir el guanyador d'una partida d'escacs fent servir tant informació prèvia dels jugadors com característiques derivades de l'estat del tauler després d'un nombre determinat de moviments.

Pel que fa a les característiques conegudes prèviament al començament de la partida, s'ha observat que, amb només aquesta informació, els models ja són capaços d'obtenir aproximadament un 66% d'encerts. Aquest resultat és prou remarcable, ja que amb dades limitades els models ja són capaços de predir correctament dues de cada tres partides, de forma molt estable.

D'altra banda, els resultats obtinguts a l'apartat anterior mostren que el fet d'incloure la informació derivada dels moviments de la partida és molt important. A mesura que augmentem el nombre de moviments, els models basats en arbres (Gradient Boosting i Random Forest) milloren clarament de manera progressiva. Des d'un punt de vista més estratègic, té sentit, ja que els primers moviments marquen l'estructura de la partida i sovint permeten detectar avantatges posicionals que són predictors bons del futur guanyador. A partir dels $N \approx 30$ observem una millora significativa, i per valors més grans, l'accuracy arriba a superar el 90%.

Pel que fa als models, finalment ens hem quedat amb Gradient Boosting, ja que ha demostrat ser el més robust i eficient. Per a N = 35 obtenim els resultats finals següents:

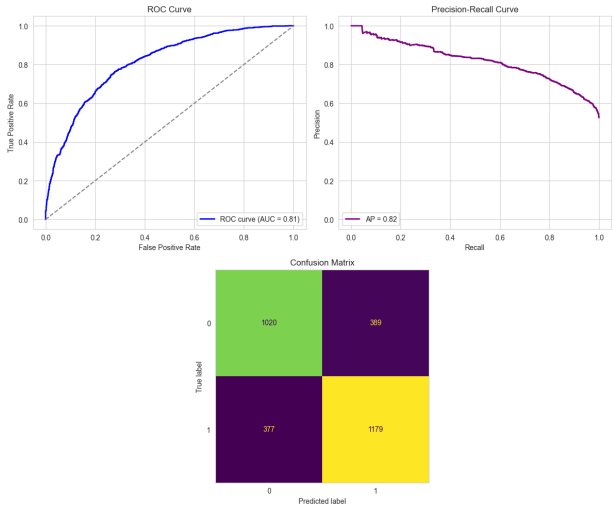

Figure 7. Resultats finals

En la primera gràfica podem veure que la corba ROC es troba sempre per sobre de la diagonal, és a dir, el nostre model té una bona capacitat per predir el guanyador d'una partida. Això ho podem recolzar amb el valor AUC (l'àrea sota la corba), que és de 0.81, un valor considerablement alt. Cal esmentar que l'accuracy que obtenim és de 0.7363, per tant, el nostre model classifica bé gairebé un 75% de les partides.

Per altra banda, podem veure també la corba de Precision-Recall, que es troba sempre a la part superior de la gràfica, el que ens indica, que tenim una Precisió i un Recall bastant alts. L'Average Precision Score és de 0.82, la qual cosa també ens indica que tenim un bon model.

Finalment, amb la matriu de confusió observem que el model encerta un nombre molt elevat tant de positius com de negatius, però també hi ha bastants casos on no es prediu correctament el guanyador. Per tant, tot i que tenim un model bo, encara es podria millorar.

Cal esmentar que ens basem en un conjunt de dades de jocs en línia, de manera que alguns resultats poden estar afectats per factors externs al joc, com problemes de connexió, interrupcions o abandonaments imprevistos. Aleshores, tot i que el model és millorable, aquesta millora té un cert límit.

En conclusió, aquest treball demostra que la combinació d'informació prèvia dels jugadors i característiques posicionals del tauler ofereix una molt bona base per predir l'evolució d'una partida d'escacs. Els resultats mostren que, tot i que les dades prèvies a la partida ja proporcionen una base sòlida, afegir informació estratègica és el que realment mostra una millora en els models per assolir rendiments molt elevats. D'aquesta manera, el projecte mostra com l'enginyeria de variables i els models basats en arbres poden capturar la complexitat d'un domini tan complex com ho són els escacs.

Els models desenvolupats podrien utilitzar-se com a eines d'anàlisi per a jugadors, permetent avaluar una partida sense necessitat d'un programa especialitzat a analitzar partides d'escacs. Només fem servir estadístiques dels jugadors i característiques posicionals simples, per tant, és un sistema molt més senzill i computacionalment eficient. D'altra banda, el sistema podria utilitzar-se per detectar patrons de rendiment anòmal entre jugadors, útil en contextos de detecció de trampes o comportaments irregulars.

Finalment, aquest treball és extrapolable a molts altres àmbits on es disposa tant d'informació prèvia com d'una seqüència d'esdeveniments (per exemple, predicció d'esdeveniments esportius, evolució de processos industrials). D'aquesta manera, en aquest treball s'ha vist que, amb un disseny adequat de les característiques i models adients, és possible capturar relacions complexes sense necessitat de recórrer a sistemes molt més costosos com els motors d'escacs basats en recerca i heurístiques avançades.